数据库原理

Database Principle

陶宏才，李晓航

西南交通大学

计算机与人工智能学院

第四章 SQL结构化查询语言

学习目的和要求

- 定义子语言
- 操纵子语言
- Transact-SQL语言



4.1 SQL语言特点及分级

一. SQL语言的特点

① DDL：SQL子集，创建、修改和删除数据库、表和视图；定义ICs；非标准索引的创建与删除。

② DML：SQL子集，插入、删除、修改和查询表中数据。

③ 嵌入(Embedded)与动态(Dynamic) SQL：嵌入SQL允许SQL代码从宿主语言(Host Language)程序中调用；动态SQL允许在运行时(run-time)构造并执行。

④ Security：控制用户访问DB对象的机制。

⑤ Transaction Management：相应命令，显式控制事务执行。

⑥ Client-Server Execution & Remote DB Access：客户应用程序如何连接DB和如何访问DB中的数据。

二. SQL-92的分级

① Entry SQL：接近SQL-89。

② Intermediate SQL：包含SQL-92近一半的新特点。

③ Full SQL：完全版SQL。



4.2 定义子语言DDL

回答如下问题:

1. 定义子语言的三个命令关键字?
2. 数据库定义?
3. 表定义?
4. 视图定义?
5. 索引定义?



一.定义子语言的三个命令关键字

(1) Create

(2) Drop

(3) Alter

说明：

① 各个命令关键字后应紧跟所要定义的对象关键字，如数据库为DATABASE、表为TABLE、视图为VIEW、索引为INDEX。

例如：CREATE DATABASE、DROP DATABASE、ALTER DATABASE等等。

② SQL SERVER中可用此三命令的数据库对象有：DATABASE、TABLE、VIEW、INDEX、TRIGGER、PROCEDURE、RULE、DEFAULT、FUNCTION。

③ SQL SERVER中，有几个数据库对象没有ALTER命令，如：INDEX、RULE、DEFAULT没有ALTER。

二.定义数据库

1.数据库及日志

数据库：在中/大型数据库系统中，数据库是一个存储空间，用于存放数据库中的数据库对象，包括表、视图、索引、存储过程、触发器、与数据库安全性有关的控制机制以及其它对象等。

日志（Log）：是数据库故障恢复的重要手段和方法。用于记录对数据库的各种操作及所涉及的相关数据，实际上也需要一个存储空间。为安全起见，一般与数据库分开存放。

2. 创建数据库（CREATE DATABASE）

Create Database命令语法:

```
CREATE DATABASE 数据库名
[ ON
  [ < filespec > [ ,...n ] ]
  ]
  [ LOG ON { < filespec > [ ,...n ] } ]
```

BFN符号约定:

::= 定义语法元素。

[ ] 表示其中的内容可选。

< > 表示其中是语法元素的名称。

| 表示必选其一。

{ } 表示必选其中的一项或多项。

[, ...n] 可按其前的样式重复。

< >、{ }和...SQL语句中不能出现。

```
< filespec > ::=
        [ PRIMARY ]
        ( [ NAME = 逻辑名 , ]
          FILENAME = 'OS文件的路径及名字'
          [ , SIZE = 文件初始大小 ]
          [ , MAXSIZE = { 最大值 | UNLIMITED } ]
          [ , FILEGROWTH = 文件大小增量值 ] )
            [ ,...n ]
```

示例:

```
Create Database StuData
On Primary
    ( Name      = StuFile1,
      Filename  = `c: \ production \ data \ StuFile1.mdf',
      Size      = 10MB,
      MaxSize   = 1000MB,
      FileGrowth = 10MB),
    ( Name      = StuFile2,
      Filename  = `c: \ production \ data \ StuFile2.ndf',
      Size      = 10MB,
      MaxSize   = 1000MB,
      FileGrowth = 10%)
Log On
    ( Name      = Stulog,
      Filename  = `c: \ production \ data \ Stulog.ldf',
      Size      = 10MB,
      MaxSize   = 1000MB,
      FileGrowth = 10MB)
```

main data file

第1个数据文件

第2个数据文件

next data file

日志文件

log data file

说明:

① 主数据文件扩展名为.mdf;

② 次数据文件扩展名为.ndf;

③ 日志文件扩展名均为.ldf。



3. 修改数据库（ALTER DATABASE）

Alter Database命令语法:

```
ALTER DATABASE 数据库名
{  ADD FILE < filespec > [ ,...n ]
 | MODIFY FILE < filespec >
 | REMOVE FILE  逻辑文件名
 | ADD LOG FILE < filespec > [ ,...n ]
 | MODIFY NAME = 新数据库名
}
```

对数据文件可增删改

对日志文件只能增加

```
 <filespec>::=
            ( NAME = 逻辑文件名
              [ , NEWNAME = 新逻辑文件名 ]
              [ , FILENAME = 'OS文件的路径及名字' ]
              [ , SIZE = 文件的初始大小 ]
              [ , MAXSIZE = 最大的文件尺寸
              [ , FILEGROWTH = 文件大小增量 ] )
```

4. 删除数据库（DROP DATABASE）

Drop Database命令语法:

DROP DATABASE 数据库名[,...n]

5. MS SQL SERVER中与数据库有关的系统存储过程

① sp_helpdb [数据库名]

② sp_renamedb 数据库旧名，数据库新名



三.定义表

1.创建表（CREATE TABLE）

Create Table 命令语法:

```
CREATE TABLE [ 数据库名.[ 架构名 ].|架构名.] 表名
({ < 列定义 >
  | 列名 AS 列计算表达式
  | < 表级约束> }
  | [ { PRIMARY KEY | UNIQUE } [ ,...] ]
 )

<列定义> ::=
    { <列名><列类型> [NULL|NOT NULL] }
    [ [ DEFAULT 常数表达式]
      | [ IDENTITY [ ( 初值 , 步长 ) [ NOT FOR REPLICATION ] ] ] ]
    [ < 列级约束 > ] [ ... ]
```

在当前DB为其它DB创建表

此情形时，”.”在[ ]外面

此情形时，”.”在[ ]里面

计算列，可定义ERM中的导出属性

表复制时，Identity不起作用

其值按初值和步长自动生成，如identity(1,1)则值为1,2,3,……；如identity(1000,-1)则为1000,999,998,……

附：Sql Server中的架构（Schema）

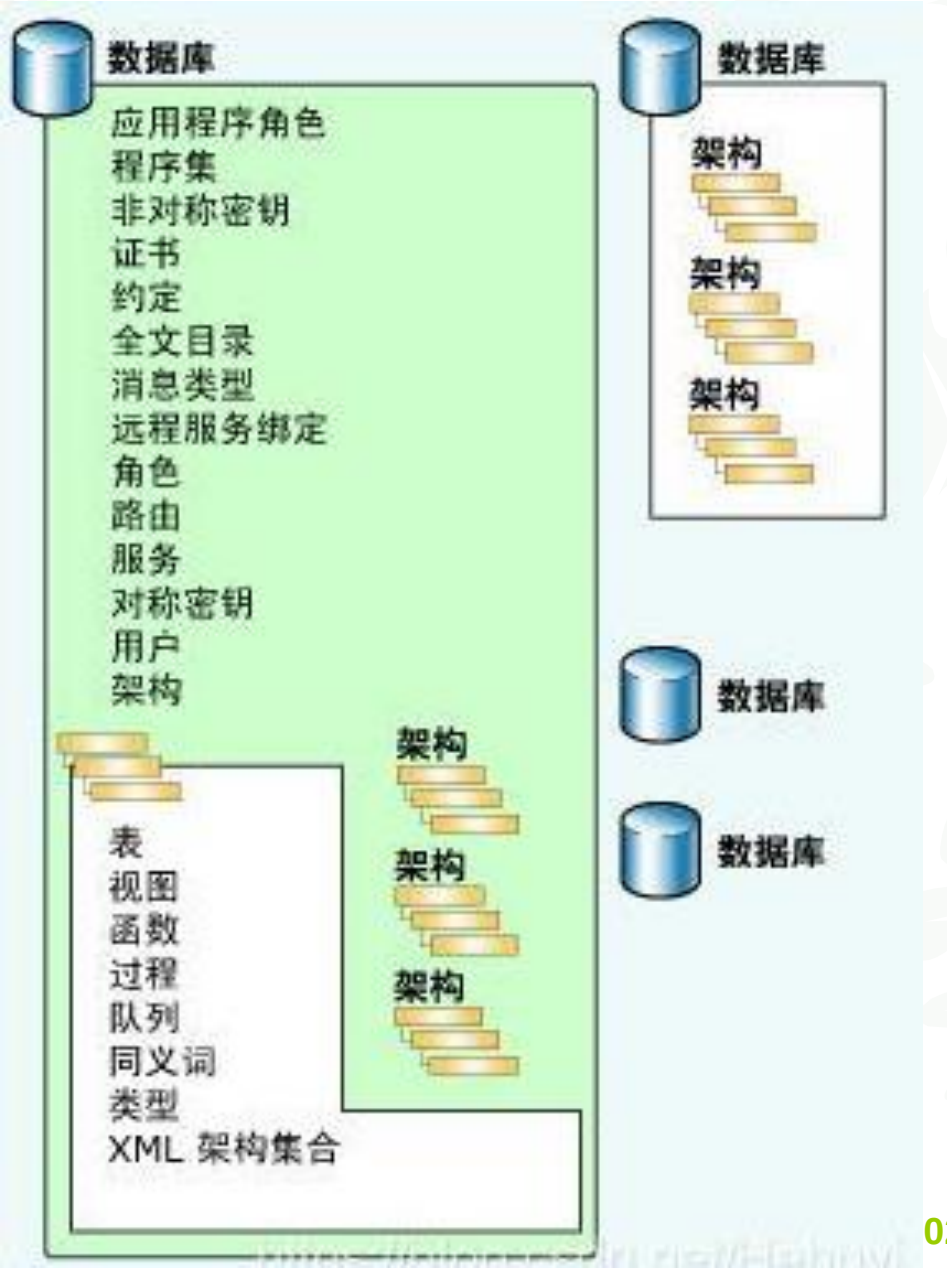
架构（Schema）是一个命名的数据库对象的容器，每个数据库对象（视图、表、存储过程、函数等）都属于一个架构。

架构将数据库对象分组为单独的命名空间，不同的架构中可以出现重名的对象。因此，访问一个数据库对象时，通常应该是两段式名称：<架构名>.<对象名>。

如果在访问数据库对象时不加<架构名>，则系统会使用默认的<dbo>架构。

可以对架构应用安全规则，安全规则将由架构中的所有对象继承。如果设置了对架构的访问权限，则当新对象添加到架构时，新对象会自动应用这些权限。用户不从架构继承权限；架构权限仅由架构中包含的数据库对象继承。

附：Sql Server中的架构（Schema）



<列级约束> ::=[CONSTRAINT 约束名]
{ [NULL | NOT NULL]
| [{ PRIMARY KEY | UNIQUE } [CLUSTERED | NONCLUSTERED]]
| [[FOREIGN KEY]
REFERENCES 参照表 [(参照列)]
[ON DELETE { CASCADE | NO ACTION }]
[ON UPDATE { CASCADE | NO ACTION }]]
| CHECK (逻辑表达式)
}

从表→主表方向

主表→从表方向

<表级约束 > ::= [CONSTRAINT 约束名]
{ [{ PRIMARY KEY | UNIQUE }
[CLUSTERED | NONCLUSTERED] {(列名[ASC | DESC] [,...])}
| FOREIGN KEY [(列名 [,...])]
REFERENCES 参照表 [(参照列 [,...])]
[ON DELETE { CASCADE | NO ACTION }]
[ON UPDATE { CASCADE | NO ACTION }]
| CHECK (条件表达式)
}

2. 修改表（ALTER TABLE）

Alter Table 命令语法:

```
ALTER TABLE 表名
{  [ ALTER COLUMN 列名  { 新数据类型 [ NULL | NOT NULL ] }]
   | ADD { [ < 列定义 > ] | 列名 AS 计算表达式 } [ ,...]
   | [ WITH CHECK | WITH NOCHECK ]
     ADD { <表级约束> } [ ,... ]
   | DROP { [ CONSTRAINT ] 约束名
          | COLUMN 列名 } [ ,... ]
   | { CHECK | NOCHECK } CONSTRAINT { ALL | 约束名 [ ,... ] }
   | { ENABLE | DISABLE } TRIGGER { ALL | 触发器名 [ ,...n ] }
}
```

对列可增加、修改

对列的删除

对约束的增加、删除

使约束起作用(Check)或不起作用(Nocheck)

使触发器使能(Enable)或失能(Disable)

3. 删除表（DROP TABLE）

Drop Table命令语法:

```
DROP TABLE 表名
```

四.定义视图

1.创建/修改视图（CREATE/ALTER VIEW）

Create/Alter View 命令语法:

CREATE/ALTER VIEW [架构名.] 视图名 [(视图列表)]
AS
SQL 查询语句
[WITH CHECK OPTION]

2.删除视图（DROP VIEW）

Drop View 命令语法:

DROP VIEW 视图名



五.定义索引

1. 索引

概念: 索引是关于数据位置信息的关键字表。

优点: 加快查询速度。缺点: 降低增删改的速度。

类型: 聚簇(Clustered)索引、非聚簇(Nonclustered)索引和唯一(Unique)索引。

聚簇索引: 表中数据与索引存储在相邻物理空间，且表中行的物理顺序与索引顺序保持一致。每张表最多只能建一个聚簇索引。

唯一索引: 被索引的列不能有相同值出现。

索引建立原则:

(1) 为数据量大的表建立索引;

(2) 一张表所建索引个数应适量;

(3) 掌握建立索引的时机;

(4) 优先建立主键列的索引。

2. 创建索引（CREATE INDEX）

Create Index 命令语法:

CREATE [UNIQUE] [CLUSTERED | NONCLUSTERED] INDEX 索引名
ON {表名 | 视图名}(列名[ASC | DESC] [,...])

3. 删除索引（DROP INDEX）

Drop Index 命令语法:

DROP INDEX 索引名



4.3 操纵子语言DML

回答如下问题:

1.操纵子语言的四个命令关键字?

2.数据插入?

3.数据修改?

4.数据删除?

5.数据查询?



一.操纵子语言的四个命令关键字

(1) Insert

(2) Update

(3) Delete

(4) Select

说明：

① 以上四个命令，均是针对数据的操作。具体说来，其操纵的对象是：表以及满足条件的视图（如：基于一张表的视图等）。

② 在定义了约束或限制的情况下，数据的“增删改”操作必须遵守相关的限制条件。

二.数据插入（INSERT）

1. 数据插入命令基本语法

INSERT [INTO] 表名或视图名 [(列名表)] <数据值>

说明：<数据值>的写法决定具体的插入方式。

2. 数据插入的两种方式

(1) 插入一行/多行（高版本）

INSERT [INTO] 表名或视图名 [(列名表)]

VALUES (列值表) [,...*n*]

(2) 插入一行或多行

INSERT [INTO] 表名或视图名 [(列名表)]

SELECT子句

三.数据修改（UPDATE）

1.数据修改命令的语法

```
UPDATE
{
    表名 [WITH (<表更新选项>[...n])]
  | 视图名
}
SET
{ 列名 = { 表达式 | DEFAULT | NULL } } [,...n]
{{ [FROM { <源表> } [,...n]]
   [WHERE  <条件表达式>] }
}

<表更新选项> ::= { PAGLOCK | READCOMMITTED | ROWLOCK |
                   TABLOCK | TABLOCKX | UPDLOCK }
```

页级锁/页锁

行级锁/行锁

读提交

表级锁/表锁

表级独占锁

更新锁

```
<源表>::=
        表名[[AS]别名][WITH(<表更新选项>[,...n])]
      |视图名[[AS]别名]
      |<表连接>

<表连接>::=
        <源表><连接类型><源表>ON<条件表达式>
      |<源表>CROSS JOIN<源表>
      |<表连接>

<连接类型>::=
        [INNER|{{LEFT|RIGHT|FULL}[OUTER]}] JOIN
```

2. 数据修改示例

示例1：简单的更新

```
UPDATE   publishers
SET   city = 'Atlanta',   state = 'GA'

UPDATE   titles   SET price = price * 2
```

示例2：带WHERE子句的更新

```
UPDATE authors
SET state = 'PC', city = 'Bay City'
WHERE state = 'CA' AND city = 'Oakland'
```

示例3：利用其他表数据的更新

```
UPDATE titles
SET ytd_sales = (  SELECT SUM(qty)
                   FROM sales
                   WHERE sales.title_id = titles.title_id AND
                         sales.ord_date IN ( SELECT MAX(ord_date)
                                             FROM sales )
                )
FROM titles, sales
```

聚集函数

相关子查询

三.数据删除（DELETE）

1. 数据删除命令的语法

```
DELETE  [ FROM ]
{   表名 [WITH ( < 表更新选项 > [ ...n ] )]
  | 视图名
}
[ FROM { < 源表 > } [ ,...n ] ]
[ WHERE { < 条件表达式 > } ]
```

2. 分别基于SQL-92、T-SQL的数据删除命令示例

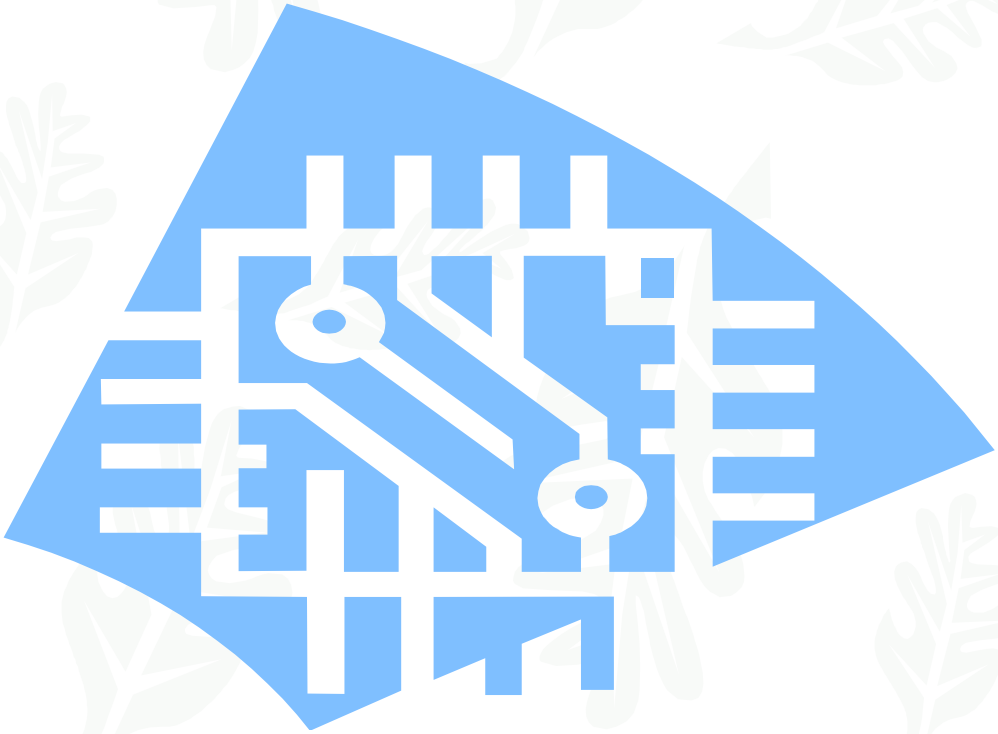
(1) SQL-92的写法

```
DELETE FROM titleauthor
WHERE title_id IN ( SELECT title_id
                    FROM titles
                    WHERE title LIKE '%computers%')
```

(2) T-SQL的写法

内连接的条件

```
DELETE titleauthor
FROM  titleauthor   INNER JOIN   titles
        ON titleauthor.title_id = titles.title_id
WHERE titles.title LIKE '%computers%'
```



四.数据查询（SELECT）

Select 列
From 表
Where 条件



结果集
{<……>, <……>, <……>, ……,
<……>, <……>, <……>}



如何对结果集更好地展示呢？

按升序排 ↓　　按降序排 ↓

省/地区	出版社	书名	单价(元)	库存(本)
北京	清华大学出版社	数据库原理及设计	32.00	1000
北京	电子工业出版社	计算机网络	56.00	890
北京	北京大学出版社	数据库系统原理	48.00	960
		按地区分组统计 →	平均价：45.33	库存量：2850
四川	西南交通大学出版社	数据库原理与应用设计	28.00	200
四川	四川大学出版社	网络安全原理	43.00	800
四川	电子科技大学出版社	TCP/IP协议	35.00	200
		按地区分组统计 →	平均价：35.33	库存量：1200
云南	云南出版社	WEB网页设计	33.00	1100
		按地区分组统计 →	平均价：33.00	库存量：1100
		汇总统计 →	总平均价：39.29	总库存量：5150

1. 查询命令语法

```
SELECT 查询列表
[ INTO 新表名 ]
FROM <源表>
[ WHERE 条件表达式 ]
[ GROUP BY 分组表达式 ]
[ HAVING 组内数据条件表达式 ]
[ ORDER BY 排序表达式 [ ASC | DESC ] ]
[ COMPUTE
        { { AVG | COUNT | MAX | MIN | SUM } ( 表达式 ) } [ ,...n ]
        [ BY expression [ ,...n ] ]
]
```

分组

排序

总计

说明：查询由子句构成，如：SELECT子句、INTO子句、FROM子句、WHERE子句、GROUP BY子句、HAVING子句、ORDER BY子句、COMPUTE子句等。

子句执行顺序：FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY。

2. SELECT子句

SELECT [ALL | DISTINCT] < select_list >

< select_list > ::=

{ * | { 表名 | 视图名 | 表别名 }.* | { 列名 | 表达式 } [[AS] 列别名] | 列别名 = 表达式 } [,...n]



说明：

(1) ALL表示重复行可出现于结果中，DISTINCT则相反，ALL为缺省情况；

(2) *表示FROM子句的表、视图中的所有列均出现于结果中，其顺序遵照表、视图中的顺序。

(3) 表达式中可用聚集函数。

3. INTO子句

作用：从已有表结构创建一新表，并将查询的结果数据插入其中。

语法：[INTO 新表名]

说明：

(1) SELECT.....INTO不能与COMPUTE一起使用；

(2) 可用SELECT.....INTO创建一张与FROM子句中的表具有相同结构的、名字不同的新表。如果不需要数据，可令WHERE子句条件为永假（如1 = 2）即可。

4. FROM子句

作用：指定在SELECT、DELETE及UPDATE语句中所用的表、视图及表连接等。

语法：同前面DELETE及UPDATE中的FROM语法。

5. WHERE子句

作用：指定限定行的查询条件。

算术比较符：=、<、>、<=或!>、>=或!<、<>或!=

逻辑运算符：AND、OR、NOT

特殊运算符：IN、NOT IN、BETWEEN...AND...、LIKE、NOT LIKE、IS NULL、IS NOT NULL、SOME|ANY、ALL、EXISTS、NOT EXISTS。

（1）LIKE

① %（匹配任意一串字符）、_（匹配任意一个字符）、[ ]（取其中任意单个字符）、[^]（不能取其中所列字符）。

②如希望%、_以常规字符出现，则应用[ ]将其括起或用转义符 “$”，如：like “70$%” 或 like “70[%]” ===> 70%。

③ ^应与[ ]联用，如：[^a–f]或[^abcdef]表示a–f这几个字母不能出现。

（2）SOME|ANY

语法：表达式{=|<>|!=|>|>=|!>|<|<=|!<}
{SOME|ANY}(子查询)

示例1：
```
Select * From A
Where A.a = SOME (Select b From B)
```

示例2：
```
Select * From A
Where 5 <> ANY (Select b From B)
```

(3) ALL

语法：表达式{=|<>|!=|>|>=|!>|<|<=|!<}
ALL (子查询)

示例：Select * From A
Where A.a > ALL (Select b From B)

(4) [NOT] EXISTS

语法：[NOT] EXISTS (子查询)

含意：检查子查询是否有结果返回，如有结果则为TRUE；如返回NULL则FALSE。如带NOT则相反。

示例1：Select * From A
Where EXISTS (Select b From B)

示例2：Select * From A
Where NOT EXISTS (Select b From B)

(5) [NOT] IN

语法：测试表达式 [NOT] IN
(
子查询
| 表达式 [,...n]
)

含意：检查测试表达式的值是否在子查询或列表中，是则TRUE，否则FALSE；如带NOT则相反。

示例1：
```
Select * From A
Where A.a IN (Select b From B)
```

示例2：
```
Select * From A
Where 8 NOT IN (Select b From B)
```

6. 聚集函数

作用：用于对数据集合进行统计，常见的统计函数：总和、平均值、最大值、最小值、行数（个数）。

用法：一般用于SELECT子句、HAVING子句、ORDER BY、COMPUTE子句中。

函数形式及功能：

COUNT(*)	返回结果的行数，含NULL行和重复行
COUNT(DISTINCT|ALL 表达式)	返回非空表达式值的行数
MAX (DISTINCT|ALL 表达式)	返回非空表达式值的最大值
MIN (DISTINCT|ALL 表达式)	返回非空表达式值的最小值
SUM (DISTINCT|ALL 表达式)	返回非空表达式值的总和
AVG (DISTINCT|ALL 表达式)	返回非空表达式值的平均值

7. GROUP BY子句与HAVING子句

GROUP BY子句作用：用于对查询的结果数据集合进行“分组”或“分组统计”，如对各个分组求：总和、平均值、最大值、最小值、行数。

GROUP BY子句用法：一般与SELECT子句的聚集函数联用。

HAVING子句作用：用于对分组数据集合的再筛选。

HAVING子句用法：一般与GROUP BY联用。

示例：按出版商编号分组查询至少出版有4本书籍的出版商及其销售总量：

```
SELECT pub_id AS '出版商编号',
        SUM(ytd_sales) AS '销售总量'
FROM titles
GROUP BY pub_id
HAVING COUNT(*) > 3
```

说明：

(1) 要求SELECT子句中的列，除了使用聚集函数的列之外，其余各列都必须出现在Group By子句中；

(2) 注意WHERE、GROUP BY及HAVING三个子句的执行顺序及含意。WHERE用于对FROM子句结果设置过滤条件；GROUP BY用于对WHERE子句的结果分组；HAVING则对GROUP BY分组的结果再过滤。

8. ORDER BY子句

作用：用于对结果集进行排序。

语法：ORDER BY { 排序表达式 [ASC | DESC] } [,...n]

说明：ORDER BY中的列一般应在SELECT子句中。

9. COMPUTE子句与COMPUTE BY子句（SQL2008后已不再支持）

概念：COMPUTE用于对查询的结果数据集合最后进行“汇总”，如果带有BY，则还可进行分组汇总。COMPUTE BY可以与COMPUTE联用。

ORDER BY与COMPUTE BY和COMPUTE综合示例

```
SELECT type AS '类型', pub_id AS '出版商编号', price AS '价格'
FROM titles   WHERE type = 'psychology'
    /* 按type、pub_id和price排序 */
ORDER BY type, pub_id, price
    /* 按type和pub_id分组统计价格之和 */
COMPUTE SUM(price) BY type, pub_id
COMPUTE SUM(price)      /* 汇总统计总价 */
```

说明： COMPUTE BY应与ORDER BY联用，且COMPUTE BY后的表达式应与ORDER BY后的表达式（或其子集）内容与顺序一致。

10. 嵌套子查询（Nested Sub-Query）

概念：一个查询语句的查询结果作为另一个查询语句的条件，这样的SELECT语句为“子查询”，亦称“嵌套查询”。

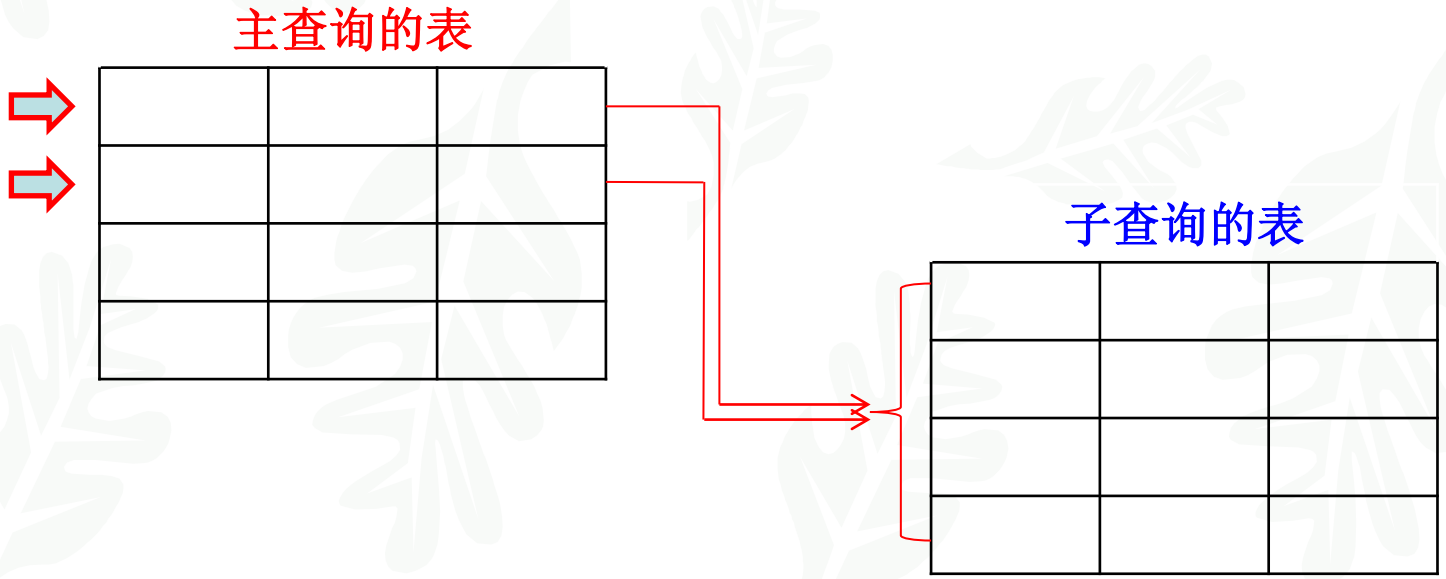
```
SELECT <查询列表>
[ INTO <新表名> ]
FROM <基表名|视图名> [ 别名 ] ……
WHERE <列名或列表达式> <比较运算符>
        (
            SELECT <查询列>
            FROM <基表名|视图名> [ 别名 ] ……
            WHERE <条件表达式>
            [ GROUP BY <分组内容>]
            [ HAVING <组内条件>]
        )
[ GROUP BY <分组内容>]
[ HAVING <组内条件>]
[ ORDER BY <排序列名> [ ASC | DESC ] ]
```

说明：

(1)子查询之中允许嵌套另一个子查询，但最多嵌套255层，并且总是从嵌套层次最深的一层开始执行，然后再执行它的直接上一层，直至完成整个查询。

(2) 子查询返回为单值时，可用“算术比较符”；返回多列或多行值，则用IN、ANY、ALL、EXISTS等运算符。

相关子查询：子查询与来自主查询（或称外查询）的列有关。意味着子查询是循环执行的，每一次处理外查询结果的一行。



示例：

```
SELECT au_lname, au_fname
FROM authors
WHERE 100 IN
        ( SELECT royaltyper
          FROM titleauthor
          WHERE titleauthor.au_ID = authors.au_id)
```

11. UNION查询

概念：将两个或多个查询的结果合并成一个结果返回。

语法：<SELECT语句> UNION [ALL] <SELECT语句>

说明：

(1) 用UNION合并的结果集应有相同的结构，即列数相同、对应列数据类型兼容。

(2) 最后结果的列名取与第一个SELECT语句返回的列名。

(3) 默认情况下，最后结果会去掉重复行；但如有ALL选项，则保留重复行。

(4) 一般，UNION个数不限，且按从左至右顺序执行。

(5) 当用UNION时，中间各个SELECT语句不能有ORDER BY和COMPUTE子句，而只能在最后一个SELECT语句后带一个ORDER BY和COMPUTE子句，它们是针对最后结果的。不过，各个SELECT语句可用GROUP BY和HAVING子句。

4.4 Transact-SQL语言

讲解内容:

1.T-SQL语言简介

2.T-SQL编程

3.T-SQL函数

4.T-SQL游标

5.T-SQL存储过程



一.T-SQL语言简介

T-SQL组成：SQL语句、函数和存储过程。

T-SQL的子语言：数据定义语言DDL（Data Definition Language）、数据操纵语言DML（Data Manipulation Language和数据控制语言DCL（Data Control Language）。

T-SQL的函数：包括系统函数和用户自定义函数。系统函数包括：聚集函数、数学函数、字符串函数、数据类型转换函数和日期函数等。

T-SQL的存储过程：由SQL语句组成，分系统存储过程和用户自定义存储过程。

二.T-SQL编程

1.注释(Comment)方式

(1) 多行时，用 /* …. */

(2) 单行时，用 --

2.变量

（1）局部变量

表示及概念：@变量名，由用户定义并使用。

局部变量定义：DECLARE @变量名 类型 [, …n]

说明：

(1) 变量一旦定义，系统自动赋NULL值。

(2) 对局部变量的赋值用SELECT（可多个变量）和SET（单个变量）。

（2）全局变量

表示及概念：@@变量名，由系统定义，用户可使用之。

作用：用于指明系统运行过程中的运行状态，用户只能引用，不能修改和定义。

常用的全局变量：

(1) @@error：返回最后一个语句产生的错误码。

(2) @@rowcount：返回语句执行后受影响的行数。

(3) @@version：SQL SERVER版本号。

(4) @@trancount：事务数。

(5) @@transtate：事务状态。

3. SELECT用法

(1) 查看表或视图的内容；

(2) 执行函数，返回值到客户端：SELECT db_name()

(3)给变量赋值

①直接赋值：Select @xxxx =2。一次也可多个。

②从表中提取值赋给变量（应保证是唯一行且类型一致）。

示例：SELECT @tp=price FROM titles T WHERE T.id='Bu1032'

(4)查看变量值：Select @变量

4.流程控制语句

(1) IF (条件关系式) ELSE [IF]：如果关系式含有SELECT语句，则该SELECT语句须用圆括号括起，其嵌套级别最多为150；

(2) BEGIN ... END：用于界定由多条SQL语句组成的语句块；

(3) WHILE ...BREAK/CONTINUE

(4) CASE语句

（A）简单CASE

```
CASE输入表达式
        WHEN 表达式1  THEN 结果表达式1
        […n]
        [ELSE  结果表达式]
END
```

示例：

```
SELECT  '分类' =
  CASE type
     WHEN 'popular_comp' THEN '普适计算'
     WHEN 'mod_cook' THEN '现代烹饪'
     WHEN 'business' THEN '商务'
     WHEN 'psychology' THEN '心理学'
     WHEN 'trad_cook' THEN '传统烹饪'
     ELSE '未分类'
  END,
  CAST(title AS varchar(25)) AS '标题简要',  price AS '价格'
FROM titles
WHERE price IS NOT NULL
ORDER BY type, price
COMPUTE  AVG(price)  BY type
```

别名=表达式

表达式 AS 别名

（B）搜索CASE

```
CASE
        WHEN 布尔表达式1  THEN 结果表达式1
        […n]
        [ELSE  结果表达式]
END
```

示例：

```
SELECT    '价格分类' =
  CASE
    WHEN price IS NULL THEN '未定价'
    WHEN price < 10 THEN '价格适中'
    WHEN price >= 10 and price < 20 THEN '价格较高'
    ELSE '价格昂贵!'
  END,
      CAST(title AS varchar(20)) AS '标题简要'
FROM titles
ORDER BY price
```

(5) GOTO ...：跳到用户定义的标号处；

(6) RETURN：无条件退出；

(7) IF EXISTS / IF NOT EXISTS（查询语句）：判断查询是否有数据返回

(8) WAITFOR：延迟某段时间

语法：WAITFOR { DELAY 日期时间格式的时间值

| TIME 时间 | …}

示例：WAITFOR DELAY '00:30:00‘ /* 延迟30分钟 */

5. 信息显示

(1) Print '…..'

(2) Print @msg

(3)格式化显示，用参数嵌套形式加入不同的值：

Print '…. %1! ….. %2! ….', @参数1，@参数2，…..

6. RAISERROR

用途：系统有很多系统信息及其代码（代码值在5万以内），用户也可用RAISERROR自定义错误信息及其代码（其代码值应在50000以上）。

语法：

Raiserror ({错误代码 | '用户定义格式化错误信息'
| 字符型局部变量}
{,严重级别, 状态}[, 参数 [,...n]])

说明：

(1) 严重级别（程度）取值0~25，其中用户可指定0~18，其它由管理员指定。

(2) 状态值取0~255，如多个位置引发同样的用户定义错误，可用状态值找到引发错误的代码段。

(3) 此函数语法与SQL Server 2000及以前版本的参数有差别。

三.T-SQL的函数

1. 字符串操作

(1) 连接操作： +

(2) convert函数——显式类型转换

语法：convert(类型符，表达式[，style])

作用：将表达式按指定风格（style）转换成指定的类型，其中的style用于日期类型、货币类型、浮点类型的科学计数法等。

(3) 日期 ===> 字符串

示例：

```
Select tid, convert(char(10), pubdate,105)
From titles
```

style取值	输出格式
1	mm/dd/yy
2	yy.mm.dd
3	dd/mm/yy
4	dd.mm.yy
5	dd-mm-yy
101	mm/dd/yyyy
102	yyyy.mm.dd
103	dd/mm/yyyy
104	dd.mm.yyyy
105	dd-mm-yyyy

2. 有关日期的几个函数

(1) getdate()

用途：将服务器OS的时间送客户机。

示例：SELECT getdate()

(2) datename()与datepart()

datename(日期元素,日期表达式)：以字符串形式返回日期元素指定的日期的名字

datepart(日期元素,日期表达式)：以数字形式返回日期元素指定的日期的名字

日期元素及其指定返回的日期部分为：

① yy：返回日期表达式中的年year或年数。

② qq：返回日期表达式表示的季quarter或季数。

③ mm：返回日期表达式中的月month或月数。

④ dw：返回日期表达式表示的星期几day of week。

⑤ dy：返回日期表达式表示的一年中的第几天day of year

⑥ dd：返回日期表达式中的天或天数day。

⑦ wk：返回日期表达式表示的一年中的第几个星期week或星期数。

⑧ hh/mi/ss：返回日期表达式中的小时hour/分minute/秒second。

示例：

Select datename(mm,pubdate) from titiles where title_id='Bu1032'

返回June，如用datepart则返回6。

(3) dateadd()与datediff()

dateadd(日期元素，数值，日期表达式)：将数值转换成日期元素指定的部分加到日期表达式上返回。

datediff(日期元素，较早日期表达式，较晚日期表达式)：两个日期相减后，按日期元素指定部分转化后返回。

3.数学函数

函　数	示 例	结 果
abs(数值表达式)	abs(-100)	100
ceiling(数值表达式)	ceiling(99.2)	100
floor(数值表达式)	floor(99.2)	99
round(数值表达式，整数表达式)	round(66.2387,2)	66.24
exp(浮点表达式)	exp(0)	1
rand([整数])	rand(23)	
log(浮点表达式)	log(1)	0
pi()	pi()	3.141 592 65...
power(数值表达式，指数表达式)	power(2,10)	1 024
sqrt(数值表达式)	sqrt(4)	2
sin(浮点表达式),cos(浮点表达式),tan(浮点表达式)	sin(pi())	0

4. 用户自定义函数

(1)返回标量值的函数

Create Function 函数名(@变量 类型[,....])

Returns 返回值的数据类型

[AS]

Begin

函数体

Return 表达式

End

(2)返回表的函数

Create Function 函数名(@变量 类型[,....])

Returns TABLE

[AS]

Return Select查询语句

(3)执行

用户自定义函数直接调用执行。

(3)示例

① 创建

```
CREATE FUNCTION SalesByStore
(@storeid varchar(30))
RETURNS TABLE
AS
RETURN ( SELECT title, qty
         FROM sales s, titles t
         WHERE s.stor_id = @storeid and
               t.title_id = s.title_id)
```

② 执行

```
select * from SalesByStore('6380')
```

四.T-SQL游标

1. 为何要游标？

Select 列
From 表
Where 条件



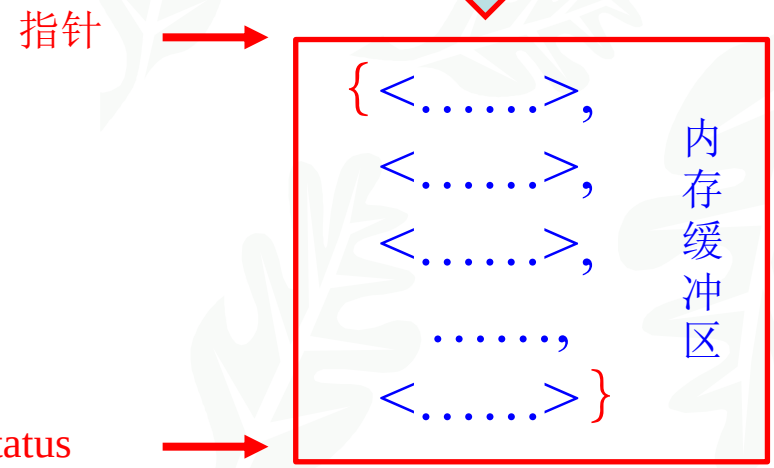
字符串型结果集

{<……>, <……>, <……>, ……,
<……>, <……>, <……>}



如果要对结果集分行分列地处理，如何办？





2. 游标生命周期

定义、Open Cursor、Fetch、Close cursor、Deallocate

文件	游标
创建文件	定义游标
打开文件	打开游标
指针指向文件开头	指针指向游标开头(非首行)
读取指针下一行，指针下移一行	Fetch
是否EOF?	@@Fetch_status
关闭文件	关闭游标
删除文件	释放游标

3.定义游标

语法： Declare 游标名 CURSOR
FOR Select语句

4.打开游标：打开后指针指向游标首位

语法：OPEN 游标名

5.FETCH

语法： Fetch 游标名 Into 局部变量列表

用法：取指针下一行，Fetch一次，指针下移一行。检查全局变量的值@@Fetch_status（0—成功；-1—失败；-2—数据行丢失，即：无数据，游标尾部）来决定如何处理。

6.Close和Deallocate

语法： Close 游标名； Deallocate Cursor 游标名

7.用游标对数据操作作

(1) Delete 表名 Where Current of 游标名

说明：删除后指针不动，下面的行自动上移。

(2) Update 表名 Set 列名=值 [, …]Where Current of 游标名

8.游标示例

(1) 游标定义

```
DECLARE  books_csr  CURSOR  FOR
    SELECT  title_id, type, price FROM titles
```

(2) 游标使用

```
Declare  @title_id char(6), @type  char(12), @price  money
Open books_csr
Fetch books_csr into @title_id, @type, @price
While  @@fetch_status != -2
Begin
        if @@fetch_status = -1
        begin
                raiserror  ('select failed', 8, 1)
                return
        end
        if @type= 'business'
                select @title_id, @type, @price, convert(money, @price*1.08)
        else if @type = 'mod_cook'
                select @title_id, @type, @price
        fetch books_csr into @title_id, @type, @price
End
```

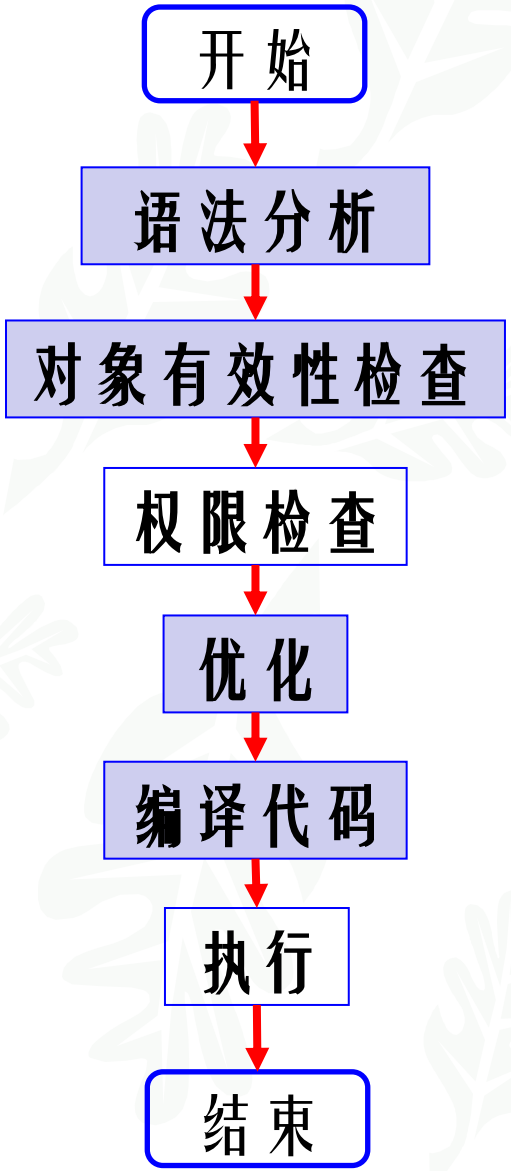
五.T-SQL存储过程

1. 为何需要存贮过程

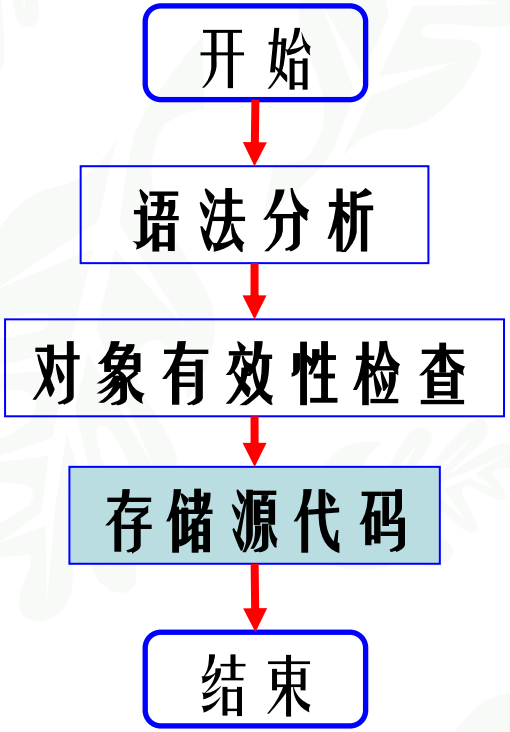
批： 可一起执行的SQL语句块。交互方式时，用GO分隔各批。

批的执行：

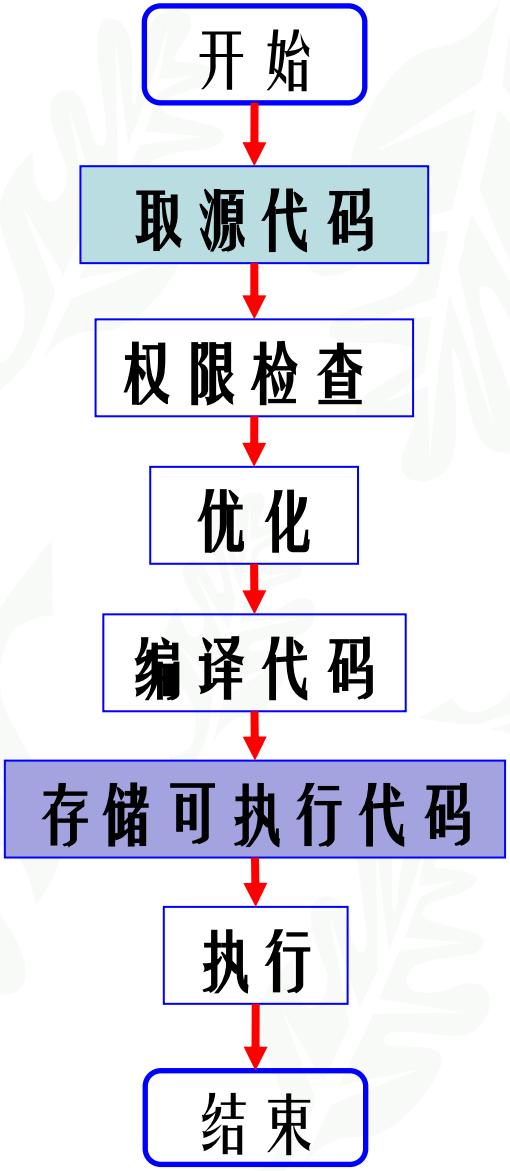
存储过程： 将经常使用的SQL语句块合为一个过程。



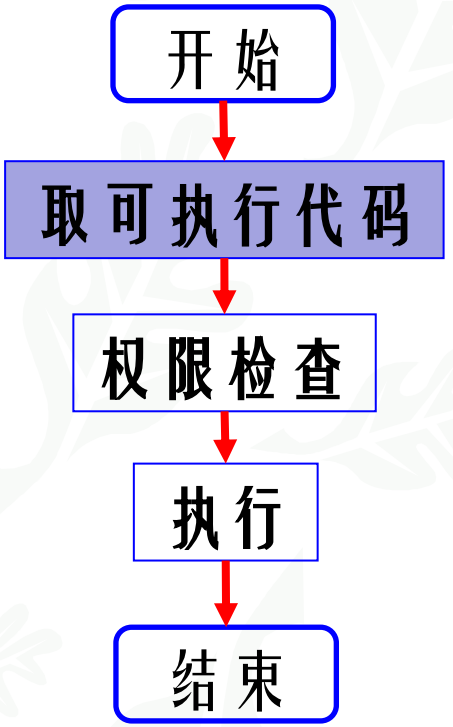
存储过程定义



第1次执行



第2次及以后执行



2. 创建存贮过程

Create Proc 过程名
(@变量 类型 [, ….]
@变量 类型 OUTPUT)
As
SQL 语句块
Return [状态整数值]

3. 调用执行

Execute，过程调用语句非首行时必写

[EXEC] [@状态接收变量=]过程名
[@参数变量=] {参数值|@局部变量} [,…n]
[,@接收输出的局部变量 OUTPUT]

说明：先定义局部变量，用于接收输出值或状态返回值。

4.查看

sp_helptext 过程名

5.改名

sp_rename 旧过程名，新名

6.删除

Drop Proc 过程名

7.对存贮过程的限制

(1)某些命令不能放在过程中

Create View/Default/Rule/Trigger/Proc等。

(2)不能在同一过程中刚删除一个DB对象，又创建同名的对象。

8.存贮过程示例

(1) 存储过程创建

```
CREATE PROC num_sales
( @book_id  char(6) ,   /* 输入参数 */
  @tot_sales  int  OUTPUT )  /* for output */
AS
        SELECT @tot_sales=sum(qty)
        FROM salesdetail
        WHERE  title_id = @book_id
RETURN 1
```

(2) 调用

```
DECLARE  @total  int
EXEC  num_sales  @book_id='pe53995' ,
                 @tot_sales=@total  OUTPUT
```

或 `EXEC num_sales 'pe53995', @total OUTPUT`

— The End —